

Relatório Técnico

Compilador da Linguagem Homi

para Home Assistant

Trabalho de Graduação – Disciplina de Compiladores

Luis Henrique Pozzebon
Rafaela da Rosa Soares

2026

Sumário

1. Introdução
2. Definição da Gramática (GLC)
3. Análise Léxica
4. Análise Sintática
5. Construção da AST
6. Análise Semântica
7. Geração de Código YAML
8. Pipeline de Compilação (main.py)
9. Exemplos Práticos
10. Estrutura de Arquivos do Projeto
11. Instruções de Execução
12. Considerações Finais

1. Introdução

1.1. Objetivo

O presente relatório descreve o projeto e a implementação de um compilador que traduz programas escritos na linguagem Homi para arquivos no formato YAML, compatíveis com o sistema de automação residencial Home Assistant. A linguagem Homi foi concebida como uma abstração de alto nível que permite ao usuário descrever regras de automação doméstica — como acionamento de luzes, verificação de estados de sensores, acionamento de interruptores e temporizações — por meio de uma sintaxe declarativa em português, dispensando o conhecimento prévio do esquema YAML nativo da plataforma-alvo.

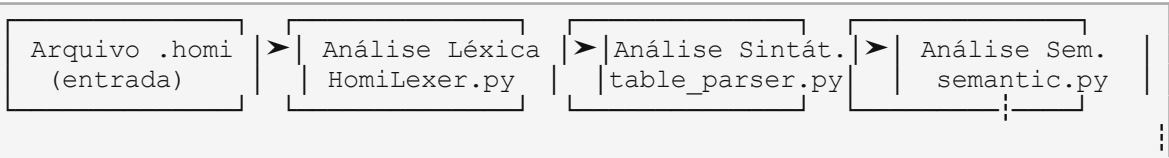
1.2. Ferramentas e Tecnologias

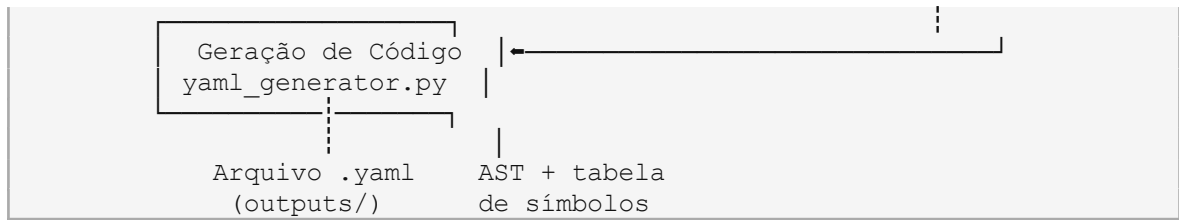
O compilador foi inteiramente desenvolvido em Python 3 e utiliza o ANTLR 4 para a fase de análise léxica. As demais fases — análise sintática, construção de AST, análise semântica e geração de código — são implementadas manualmente em Python puro.

Ferramenta / Biblioteca	Módulo / Arquivo	Papel no compilador
ANTLR 4.13.2	HomiLexer.py (gerado)	Geração automática do analisador léxico a partir de Homi.g4
Python 3 (puro)	table_parser.py	Parser LL(1) por tabela preditiva, construído manualmente
Python 3 (puro)	ast_nodes.py	Definição dos nós da AST via dataclasses
Python 3 (puro)	ast_builder.py	Visitor ANTLR alternativo para construção da AST
Python 3 (puro)	semantic.py	Análise semântica e tabela de símbolos
Python 3 (puro)	yaml_generator.py	Geração do YAML para Home Assistant
Python 3 (puro)	main.py	Orquestração do pipeline completo de compilação

1.3. Arquitetura Geral do Pipeline

O compilador adota a arquitetura clássica de múltiplas fases sequenciais. O módulo main.py orquestra a execução do pipeline e interrompe o processo ao detectar erros em qualquer fase, impedindo que erros de uma etapa se propaguem e causem resultados incorretos nas etapas subsequentes.





Cada fase comunica-se com a subsequente por meio de estruturas de dados bem definidas: o lexer produz uma lista de tokens, o parser consome os tokens e produz a AST, o analisador semântico valida a AST e exporta a tabela de símbolos, e o gerador percorre a AST para produzir o YAML final.

2. Definição da Gramática (GLC)

A linguagem Homi é definida por uma Gramática Livre de Contexto (GLC) especificada no arquivo `grammar/Homi.g4` usando a notação do ANTLR 4. A gramática é do tipo LL(1) — livre de contexto, sem ambiguidades e com apenas um token de lookahead — o que permite a construção de um parser descendente determinístico por tabela preditiva.

2.1. Produções da GLC

A seguir, as regras de parser do arquivo `Homi.g4` na notação BNF. Símbolos terminais são grafados em maiúsculo (tokens) e não-terminais em minúsculo:

```
programa
    : declaracao* automacao+ EOF
    ;

declaracao
    : ENTIDADE dominio IDENT IGUAL ENTITY_ID PONTO_VIRGULA
    ;

dominio
    : LUZ | SENSOR | INTERRUPTOR | ALARME
    ;

automacao
    : AUTOMACAO STRING ABRE_CHAVE corpoAutomacao FECHA_CHAVE
    ;

corpoAutomacao
    : blocoGatilho blocoCondicoes? blocoAcoes modo?
    ;

blocoGatilho
    : QUANDO gatilho PONTO_VIRGULA
    ;

gatilho
    : ESTADO referencia FICAR valor
    ;

blocoCondicoes                                // bloco opcional
    : SE expressaoCondicao PONTO_VIRGULA
    ;

expressaoCondicao
    : termoCondicao (operadorLogico termoCondicao)*
    ;

termoCondicao
    : NAO? condicaoEstado                        // NAO e opcional
    ;

condicaoEstado
    : ESTADO referencia FICAR valor
    ;
```

```

operadorLogico
  : E | OU
  ;

blocoAcoes
  : FACA ABRE_CHAVE comando* FECHA_CHAVE
  ;

comando
  : acaoSimples PONTO_VIRGULA
  | espera PONTO_VIRGULA
  ;

acaoSimples : verboAcao referencia ;
verboAcao   : LIGAR | DESLIGAR ;
espera      : ESPERAR DURATION ;
modo        : MODO modoValor PONTO_VIRGULA ;
modoValor   : SINGLE | RESTART | QUEUED | PARALLEL ;
referencia  : IDENT | ENTITY_ID ;
valor       : STATE | STRING | NUMBER | BOOLEAN ;

```

2.2. Classificação dos Símbolos

Terminais (Tokens):

Token	Padrão / Valor	Exemplo de lexema
ENTITY_ID	[a-z_][a-z0-9_]* '.' [a-zA-Z0-9_]+	binary_sensor.movimento, light.corredor
IDENT	[a-zA-Z_][a-zA-Z0-9_]*	corredor, movimento, sala
STRING	'"' (~['"\r\n])* "'	"Corredor com movimento"
DURATION	[0-9]+ ('s' 'min' 'h')	45s, 10min, 1h
NUMBER	[0-9]+ ('.' [0-9]+)?	3, 1.5
STATE	on off armed disarmed ...	on, off, idle, running
BOOLEAN	true false	true, false
COMMENT	// ... ou # ...	# comentario

Não-terminais: programa, declaracao, dominio, automacao, corpoAutomacao, blocoGatilho, gatilho, blocoCondicoes, expressaoCondicao, termoCondicao, condicaoEstado, operadorLogico, blocoAcoes, comando, acaoSimples, verboAcao, espera, modo, modoValor, referencia, valor.

2.3. Observações sobre a Gramática

- O não-terminal blocoCondicoes admite derivação para a cadeia vazia (ϵ), tornando o bloco de condições facultativo. Igualmente, o não-terminal modo é opcional.

- O operador NAO em termoCondicao também é opcional (NAO?), permitindo negação de qualquer termo de condição.
- A gramática não apresenta ambiguidades: cada sequência de tokens possui uma única derivação possível.
- O token ENTITY_ID tem prioridade sobre IDENT no ANTLR pelo ordenamento das regras: qualquer sequência contendo ponto (ex: light.corredor) é reconhecida como ENTITY_ID.
- Comentários (// e #) e espaços em branco são descartados com a diretiva -> skip, não chegando ao parser.

3. Análise Léxica

3.1. Visão Geral

A fase de análise léxica é implementada pelo módulo HomiLexer.py, gerado automaticamente pelo ANTLR 4 a partir das regras léxicas definidas em grammar/Homi.g4. O analisador léxico (scanner) percorre o fluxo de caracteres do código-fonte e o segmenta em uma sequência de tokens — as unidades léxicas mínimas com significado gramatical. Internamente, o ANTLR constrói um autômato finito determinístico (AFD) serializado como ATN (Augmented Transition Network) a partir das expressões regulares declaradas.

3.2. Geração do Lexer

O lexer é gerado pelo seguinte comando, disponível via make generate:

```
java -jar antlr-4.13.2-complete.jar \  
  -Dlanguage=Python3 \  
  -visitor \  
  grammar/Homi.g4 \  
  -o generated/grammar
```

A flag -visitor instrui o ANTLR a gerar também HomiVisitor.py e HomiListener.py, interfaces para percorrer a parse tree. Os arquivos gerados não devem ser editados manualmente.

3.3. Especificação dos Tokens (regras léxicas do Homi.g4)

```
// Palavras-chave da linguagem  
ENTIDADE: 'entidade'; AUTOMACAO: 'automacao'; QUANDO: 'quando';  
ESTADO: 'estado'; FICAR: 'ficar'; FACA: 'faca';  
LIGAR: 'ligar'; DESLIGAR: 'desligar'; ESPERAR: 'esperar';  
MODO: 'modo'; SE: 'se';  
E: 'e'; OU: 'ou'; NAO: 'nao';  
  
// Domínios de entidade  
LUZ: 'luz'; SENSOR: 'sensor'; INTERRUPTOR: 'interruptor'; ALARME:  
'alarme';  
  
// Modos de execução  
SINGLE: 'single'; RESTART: 'restart'; QUEUED: 'queued'; PARALLEL:  
'parallel';  
  
// Estados predefinidos do Home Assistant  
STATE : 'on' | 'off' | 'disarmed' | 'armed' | 'charging' | 'idle' |  
'running' ;  
BOOLEAN : 'true' | 'false' ;  
  
// Pontuação  
IGUAL: '='; ABRE_CHAVE: '{'; FECHA_CHAVE: '}'; PONTO_VIRGULA: ',';  
  
// Literais  
DURATION : [0-9]+ ('s' | 'min' | 'h') ;  
NUMBER : [0-9]+ ('.' [0-9]+)? ;
```



```
ENTITY_ID: [a-z_][a-z0-9_]* '.' [a-zA-Z0-9_]+ ;
IDENT    : [a-zA-Z_][a-zA-Z0-9_]* ;
STRING   : '"' (~['"' '\r' '\n'])* '"' ;

// Ignorados pelo scanner
COMMENT  : ('/' ~[\r\n]* | '#' ~[\r\n]*) -> skip ;
WS       : [ \t\r\n]+ -> skip ;
```

3.4. Integração com o Compilador

Em main.py, o lexer é instanciado com um FileStream e um listener de erros personalizado. O método token_stream.fill() materializa todos os tokens antes da análise sintática. O parser LL(1) filtra os tokens de canal padrão (WS e COMMENT já foram descartados pelo skip):

```
input_stream = FileStream(input_file, encoding='utf-8')
lexer = HomiLexer(input_stream)
lexer.removeErrorListeners()
lexer.addErrorListener(syntax_error_collector) # coleta erros léxicos
token_stream = CommonTokenStream(lexer)
token_stream.fill() # tokeniza o arquivo inteiro antes do parser
```

3.5. Tratamento de Erros Léxicos

O compilador substitui o listener de erros padrão do ANTLR por um SyntaxErrorCollector personalizado. Ao encontrar um caractere que não satisfaz nenhuma regra léxica, a mensagem é acumulada na lista de erros com linha e coluna precisas. Se houver qualquer erro léxico ao final do fill(), o compilador exhibe todos os erros coletados e encerra sem prosseguir para as fases seguintes.

3.6. Formato de Saída

Cada token emitido pelo analisador léxico é um objeto do ANTLR com os seguintes atributos relevantes:

Atributo	Tipo	Descrição
type	int	Código numérico do tipo do token (constante da classe HomiLexer)
text	str	Lexema original reconhecido (ex: 'light.corredor', 'on')
line	int	Número da linha em que o token foi encontrado
column	int	Posição da coluna (caractere de início do token na linha)
channel	int	Canal do token; apenas canal DEFAULT_CHANNEL chega ao parser

4. Análise Sintática

4.1. Visão Geral

A análise sintática é implementada no módulo `table_parser.py` por meio de um parser descendente não-recursivo (iterativo) baseado em tabela preditiva LL(1). Essa implementação é inteiramente manual, sem uso do HomiParser gerado pelo ANTLR, e constitui o núcleo técnico do trabalho. O algoritmo emprega uma pilha explícita e uma tabela preditiva `PARSE_TABLE` para determinar, dado o não-terminal no topo da pilha e o token de lookahead atual, qual produção deve ser expandida.

4.2. Estrutura da Tabela Preditiva

A tabela preditiva `PARSE_TABLE` é um dicionário Python indexado por pares (não-terminal, `token_de_lookahead`), cujo valor é a produção a expandir (tupla de símbolos) ou `EPSILON` para produções vazias. Exemplos representativos da tabela implementada:

```
PARSE_TABLE = {
    # Programa aceita declaracoes seguidas de automacoes
    ('programa', ENTIDADE): ('declaracao_lista', 'automacao_lista', EOF),
    ('programa', AUTOMACAO): ('declaracao_lista', 'automacao_lista', EOF),

    # declaracao_lista: repete enquanto houver 'entidade'
    ('declaracao_lista', ENTIDADE): ('declaracao', 'declaracao_lista'),
    ('declaracao_lista', AUTOMACAO): EPSILON,

    # declaracao: sintaxe completa de declaracao de entidade
    ('declaracao', ENTIDADE): (ENTIDADE, 'dominio', IDENT, IGUAL,
ENTITY_ID, PONTO_VIRGULA),

    # dominio: os quatro tipos de entidade suportados
    ('dominio', LUZ): (LUZ,),
    ('dominio', SENSOR): (SENSOR,),
    ('dominio', INTERRUPTOR): (INTERRUPTOR,),
    ('dominio', ALARME): (ALARME,),

    # condicoes_opcional: presente se houver 'se', ausente se houver
'faca'
    ('condicoes_opcional', SE): ('bloco_condicoes',),
    ('condicoes_opcional', FACA): EPSILON,

    # espera: 'esperar DURATION'
    ('espera', ESPERAR): (ESPERAR, DURATION),

    # modo_valor: quatro modos de execucao do Home Assistant
    ('modo_valor', SINGLE): (SINGLE,),
    ('modo_valor', RESTART): (RESTART,),
    ('modo_valor', QUEUED): (QUEUED,),
    ('modo_valor', PARALLEL): (PARALLEL,),
}
```

4.3. Algoritmo de Parsing por Pilha

O método `validate_by_table` implementa o algoritmo clássico de parsing LL(1) com pilha explícita:

```
def validate_by_table(self):
    stack = [EOF, 'programa'] # pilha inicializada com simbolo inicial

    while stack:
        top = stack.pop()
        lookahead = self.current().type

        if isinstance(top, int): # top e terminal
            self.match_terminal(top)
            continue

        # top e nao-terminal: consulta tabela preditiva
        production = self.PARSE_TABLE.get((top, lookahead))
        if production is None:
            self.recover_non_terminal(top, lookahead, stack)
            continue

        if production == EPSILON:
            continue # producao vazia: nao empilha nada

        for symbol in reversed(production): # empilha na ordem reversa
            stack.append(symbol)
```

4.4. Conjuntos FIRST Computados Dinamicamente

Os conjuntos FIRST de cada não-terminal são derivados automaticamente da própria `PARSE_TABLE` no método `build_first_sets`, evitando redundância e garantindo consistência entre a tabela e os conjuntos usados na recuperação de erros:

```
def build_first_sets(self):
    first_sets = {}
    for non_terminal, lookahead in self.PARSE_TABLE:
        first_sets.setdefault(non_terminal, set()).add(lookahead)
    return first_sets
```

4.5. Recuperação de Erros Sintáticos — Modo Pânico

O compilador implementa a técnica de Modo Pânico para assegurar a resiliência da análise perante entradas malformadas. Ao encontrar um token inesperado para um dado não-terminal, o parser reporta o erro e descarta tokens até encontrar um símbolo do conjunto FIRST ou FOLLOW do não-terminal atual, podendo assim detectar múltiplos erros em uma única execução:

```
def recover_non_terminal(self, non_terminal, lookahead, stack):
    expected = self.expected_tokens(non_terminal)
    self.report_current(f'nao entendi este trecho; esperava {expected}')

    follow = self.FOLLOW.get(non_terminal, {EOF})
```

```
first = self.first_sets.get(non_terminal, set())

if lookahead in follow or lookahead == EOF:
    return # já sincronizado, continua sem descartar

# descarta tokens ate sincronizar com FIRST ou FOLLOW
while self.current().type not in first and self.current().type not in follow:
    if self.current().type == EOF: return
    self.pos += 1

if self.current().type in first:
    stack.append(non_terminal) # tenta reanalisar o nao-terminal
```

O dicionário FOLLOW, declarado estaticamente no TableParser, mapeia cada não-terminal ao conjunto de tokens que podem segui-lo, permitindo que o parser identifique o ponto de sincronização correto. Essa abordagem em dois níveis (FIRST para reentrada, FOLLOW para sincronização) permite reportar múltiplos erros sintáticos em uma única execução.

5. Construção da AST

5.1. Visão Geral

Após a validação sintática sem erros, o TableParser realiza uma segunda passagem sobre os tokens para construir a AST (método `build_ast`). O parser executa as duas passagens de forma automática pelo método `parse()`: primeiro `validate_by_table`, e somente se não houver erros, `build_ast`.

5.2. Nós da AST (`ast_nodes.py`)

A AST é composta por dataclasses Python definidas em `ast_nodes.py`. Cada nó carrega o número de linha do token de origem para reporte preciso de erros nas fases seguintes:

```
@dataclass
class Program:
    declarations: List[EntityDecl]    # lista de declaracoes de entidade
    automations: List[Automation]     # lista de automacoes definidas

@dataclass
class EntityDecl:
    domain: str    # 'luz', 'sensor', 'interruptor', 'alarme'
    name: str      # nome simbolico (ex: 'corredor')
    entity_id: str # entity_id do HA (ex: 'light.corredor')
    line: int

@dataclass
class Automation:
    name: str          # alias da automacao
    trigger: StateTrigger
    condition: Optional[ConditionExpression] # None se ausente
    actions: List[Action]
    mode: Optional[str] # None se ausente
    line: int

@dataclass
class StateTrigger:
    entity: str # referencia a entidade (IDENT ou ENTITY_ID)
    value: str  # valor de estado esperado
    line: int

@dataclass
class ConditionExpression:
    terms: List[ConditionTerm]
    operators: List[str] # 'e' ou 'ou'
    line: int

@dataclass
class ConditionTerm:
    entity: str
    value: str
    negated: bool = False # True se precedido por NAO
    line: int = 0

@dataclass
class SimpleAction(Action):
```

```
verb:  str    # 'ligar' ou 'desligar'
entity: str    # referencia a entidade
line:  int

@dataclass
class DelayAction(Action):
    duration: str    # ex: '45s', '10min', '1h'
    line:      int
```

5.3. Abordagem Alternativa: AstBuilder (Visitor ANTLR)

O arquivo `ast_builder.py` implementa a mesma construção de AST utilizando o padrão Visitor sobre a parse tree gerada pelo HomiParser (parser do ANTLR). Essa implementação não é utilizada no pipeline principal, mas demonstra a abordagem convencional de integração com o ANTLR, delegando a construção da AST a métodos `visitXxx` correspondentes a cada regra gramatical. O projeto disponibiliza ambas as abordagens, evidenciando o contraste entre o parser manual e o pipeline automático do ANTLR.

6. Análise Semântica

6.1. Visão Geral

A análise semântica é implementada no módulo semantic.py pela classe SemanticAnalyzer, que percorre a AST para verificar consistências que não podem ser capturadas pela gramática livre de contexto. Enquanto as fases anteriores verificam a conformidade estrutural do programa, a análise semântica valida a coerência lógica das operações: tipos de entidade compatíveis com ações e estados, referências a entidades declaradas e unicidade de nomes.

6.2. Tabela de Símbolos

A tabela de símbolos é construída em duas estruturas complementares: symbol_table mapeia o nome simbólico de cada entidade ao seu nó EntityDecl, e entity_id_table mapeia o entity_id do Home Assistant ao mesmo nó. Isso permite resolver referências tanto por nome simbólico quanto por entity_id diretamente no código:

```
def build_symbol_table(self, program: Program):
    for decl in program.declarations:
        # registra em symbol_table e entity_id_table; detecta duplicatas
        self.check_entity_declaration(decl)

    for automation in program.automations:
        if automation.name in self.automation_names:
            self.add_error(automation.line,
                           f"automacao '{automation.name}' ja foi declarada.")
        else:
            self.automation_names.add(automation.name)
```

6.3. Verificações Realizadas

Verificação	Descrição
Entidade não declarada	Qualquer referência (gatilho, condição ou ação) a uma entidade ausente da symbol_table gera erro com número de linha
Nomes duplicados	Duas declarações de entidade com o mesmo nome simbólico ou o mesmo entity_id são detectadas
Automações duplicadas	Duas automações com o mesmo alias são reportadas
Estado inválido para domínio	O valor do gatilho/condição é validado contra TRIGGER_STATE_MAP por domínio (ex: sensor não aceita armed)
Ação incompatível com domínio	Ligar/desligar só é permitido para luz, interruptor e alarme (ACTION_DOMAIN_MAP); sensor não pode ser acionado
Duração inválida	O padrão ^([1-9][0-9]*) (s min h)\$ valida o campo esperar; valor zero ou formato incorreto gera erro

Verificação	Descrição
Operadores mistos	Uso simultâneo de 'e' e 'ou' na mesma expressaoCondicao é reportado como erro

6.4. Mapeamentos de Domínio

```
# Acoes permitidas por dominio
ACTION_DOMAIN_MAP = {
    'ligar': ['luz', 'interruptor', 'alarme'],
    'desligar': ['luz', 'interruptor', 'alarme'],
}

# Estados validos por dominio (gatilhos e condicoes)
TRIGGER_STATE_MAP = {
    'luz': ['on', 'off'],
    'interruptor': ['on', 'off'],
    'alarme': ['armed', 'disarmed', 'on', 'off'],
    'sensor': ['on', 'off', 'armed', 'disarmed', 'charging', 'idle',
'running'],
}
```

6.5. Acúmulo de Erros

O analisador adota a estratégia de acúmulo de erros: a execução nunca é abortada prematuramente. Cada violação semântica detectada é acumulada na lista `self.errors`, onde cada entrada é uma string formatada com o número da linha e a mensagem descritiva. Ao término da visitação completa da AST, o método `analyze` retorna a lista de erros, permitindo ao usuário corrigir múltiplos problemas em uma única iteração de compilação.

7. Geração de Código YAML

7.1. Visão Geral

A geração de código é implementada no módulo `yaml_generator.py` pela classe `YamlGenerator`, que percorre a AST validada e constrói o texto YAML compatível com o esquema oficial de automações do Home Assistant. A classe recebe a `symbol_table` e a `entity_id_table` do `SemanticAnalyzer` e itera sobre os nós `Automation` para produzir um arquivo YAML por automação.

7.2. Mapeamento de Domínios Homi → Home Assistant

```
DOMAIN_MAP = {
    'luz': 'light',
    'interruptor': 'switch',
    'alarme': 'alarm_control_panel',
}
```

7.3. Regras de Tradução

Construção Homi	Campo YAML gerado
automacao "Nome"	alias: "Nome"
quando estado X ficar on	trigger: [{platform: state, entity_id: ..., to: "on"}]
se estado X ficar off	condition: [{condition: state, entity_id: ..., state: "off"}]
se X e Y (AND)	Lista plana de condition: state
se X ou Y (OR)	condition: [{condition: or, conditions: [...]}]
nao estado X ficar Y	condition: [{condition: not, conditions: [...]}]
ligar tipo.id	action: [{service: tipo.turn_on, target: {entity_id: ...}}]
desligar tipo.id	action: [{service: tipo.turn_off, target: {entity_id: ...}}]
esperar 45s	action: [{delay: "45s"}]
modo restart	mode: restart

7.4. Geração de Condições Compostas

O gerador implementa lógica específica para condições compostas: se todos os operadores forem 'e', gera uma lista plana de condições de estado (AND implícito)

no Home Assistant); se forem 'ou', envolve em condition: or; e se houver NAO, gera condition: not aninhado:

```
def add_condition_expression(self, lines, condition, indent):
    if not condition.operators or set(condition.operators) == {'e'}:
        # AND implicit: lista plana de conditions
        for term in condition.terms:
            self.add_condition_term(lines, term, indent + '- ')
        return

    # OR explicit: envolve em condition: or
    lines.append(f'{indent}- condition: or')
    lines.append(f'{indent} conditions:')
    for term in condition.terms:
        self.add_condition_term(lines, term, indent + '    - ')
```

7.5. Exportação

Os arquivos YAML gerados são gravados no diretório outputs/, nomeados com base no alias da automação (espaços substituídos por underscore). O compilador exibe no terminal o caminho de cada arquivo gerado.

8. Pipeline de Compilação (main.py)

O arquivo main.py orquestra todas as fases do compilador em um pipeline com interrupção antecipada: ao detectar erros em qualquer fase, o compilador reporta as mensagens e encerra sem prosseguir para as fases seguintes.

```
def main():
    # [1/4] Analise Lexica (ANTLR)
    input_stream = FileStream(input_file, encoding='utf-8')
    lexer = HomiLexer(input_stream)
    lexer.addErrorListener(syntax_error_collector)
    token_stream = CommonTokenStream(lexer)
    token_stream.fill()
    if syntax_error_collector.errors:
        print(f'{N} erro(s) lexicos'); return

    # [2/4] Analise Sintatica + Construção de AST (parser LL(1))
    parser = TableParser(token_stream.tokens)
    ast = parser.parse() # validate_by_table() + build_ast()
    if parser.errors:
        print(f'{N} problema(s) de sintaxe'); return
    print('Sintaxe ok.')

    # [3/4] Analise Semantica
    semantic_analyzer = SemanticAnalyzer()
    semantic_errors = semantic_analyzer.analyze(ast)
    if semantic_errors:
        print(f'{N} problema(s) semantico(s)'); return
    print('Semantica ok.')
    print('Programa valido. Gerando YAML...')

    # [4/4] Geracao de YAML
    generator = YamlGenerator(
        semantic_analyzer.symbol_table,
        semantic_analyzer.entity_id_table
    )
    for automation in ast.automations:
        yaml_text = generator.generate(automation)
        filename = Path('outputs') / (automation.name.replace(' ', '_') +
            '.yaml')
        filename.write_text(yaml_text, encoding='utf-8')
        print(f'- Arquivo gerado: {filename}')
```

9. Exemplos Práticos

Esta seção demonstra o compilador em ação com os arquivos de exemplo reais do projeto, cobrindo um caso válido completo e os quatro casos de erro.

9.1. Exemplo Válido — exemplo1.homi

Automação que liga a luz do corredor ao detectar movimento (com condição e modo restart):

Código-fonte Homi (entrada):

```
entidade sensor movimento = binary_sensor.movimento;
entidade luz corredor = light.corredor;

automacao "Corredor com movimento" {
    quando estado movimento ficar on;
    se estado corredor ficar off;

    faca {
        ligar corredor;
        esperar 45s;
        desligar corredor;
    }

    modo restart;
}
```

Tokens gerados pelo analisador léxico:

Token	Valor (lexema)	Linha	Col
ENTIDADE	entidade	1	0
SENSOR	sensor	1	9
IDENT	movimento	1	16
IGUAL	=	1	26
ENTITY_ID	binary_sensor.movimento	1	28
PONTO_VIRGULA	;	1	51
ENTIDADE	entidade	2	0
LUZ	luz	2	9
IDENT	corredor	2	13
IGUAL	=	2	22
ENTITY_ID	light.corredor	2	24
PONTO_VIRGULA	;	2	38

Token	Valor (lexema)	Linha	Col
AUTOMACAO	automacao	4	0
STRING	"Corredor com movimento"	4	10
ABRE_CHAVE	{	4	35
QUANDO	quando	5	4
ESTADO	estado	5	11
IDENT	movimento	5	18
FICAR	ficar	5	28
STATE	on	5	34
PONTO_VIRGULA	;	5	36
SE	se	6	4
ESTADO	estado	6	7
IDENT	corredor	6	14
FICAR	ficar	6	23
STATE	off	6	29
PONTO_VIRGULA	;	6	32
FACA	faca	8	4
ABRE_CHAVE	{	8	9
LIGAR	ligar	9	8
IDENT	corredor	9	14
PONTO_VIRGULA	;	9	22
ESPERAR	esperar	10	8
DURATION	45s	10	16
PONTO_VIRGULA	;	10	19
DESLIGAR	desligar	11	8
IDENT	corredor	11	17
PONTO_VIRGULA	;	11	25
FECHA_CHAVE	}	12	4
MODO	modo	14	4
RESTART	restart	14	9
PONTO_VIRGULA	;	14	16
FECHA_CHAVE	}	15	0
EOF	<EOF>	16	0

AST gerada pelo parser LL(1):

```
Program(  
  declarations=[  
    EntityDecl(domain='sensor', name='movimento',  
               entity_id='binary_sensor.movimento', line=1),  
    EntityDecl(domain='luz', name='corredor',  
               entity_id='light.corredor', line=2),  
  ],  
  automations=[  
    Automation(  
      name='Corredor com movimento',  
      trigger=StateTrigger(entity='movimento', value='on', line=5),  
      condition=ConditionExpression(  
        terms=[ConditionTerm(entity='corredor', value='off',  
                              negated=False, line=6)],  
        operators=[],  
        line=6  
      ),  
      actions=[  
        SimpleAction(verb='ligar', entity='corredor', line=9),  
        DelayAction(duration='45s', line=10),  
        SimpleAction(verb='desligar', entity='corredor', line=11),  
      ],  
      mode='restart',  
      line=4  
    ),  
  ]  
)
```

Tabela de símbolos após análise semântica:

Nome Simbólico	Domínio	entity_id (Home Assistant)
movimento	sensor	binary_sensor.movimento
corredor	luz	light.corredor

YAML gerado (outputs/Corredor_com_movimento.yaml):

```
alias: "Corredor com movimento"  
  
trigger:  
  - platform: state  
    entity_id: binary_sensor.movimento  
    to: "on"  
  
condition:  
  - condition: state  
    entity_id: light.corredor  
    state: "off"  
  
action:  
  - service: light.turn_on  
    target:
```

```
    entity_id: light.corredor
  - delay: "45s"
  - service: light.turn_off
    target:
      entity_id: light.corredor
mode: restart
```

Saída do compilador no terminal:

```
$ python3 src/main.py examples/exemplo1.homi
Sintaxe ok.
Semantica ok.
Programa valido. Gerando YAML...
- Arquivo gerado: outputs/Corredor_com_movimento.yaml
```

9.2. Erro Sintático Simples — erro_sintatico.homi

Automação com ponto-e-vírgula faltando ao final do bloco quando:

```
entidade sensor movimento = binary_sensor.movimento;
entidade luz corredor = light.corredor;

automacao "Erro sintatico" {
    quando estado movimento ficar on    // <- falta ';' aqui

    faca {
        ligar corredor;
    }

    modo restart;
}
```

Saída do compilador:

```
$ python3 src/main.py examples/erro_sintatico.homi
Encontrei 1 problema de sintaxe.
- Linha 7, coluna 4: faltou ';' antes de 'faca'
```

9.3. Múltiplos Erros Sintáticos — erro_sintatico_multiplos.homi

Três erros sintáticos distintos em uma mesma automação, demonstrando a recuperação em modo pânico:

```
entidade sensor movimento = binary_sensor.movimento // <- falta ';'
entidade luz corredor = light.corredor;

automacao "Erros sintaticos multiplos" {
    quando estado movimento ficar on    // <- falta ';'

    faca {
        ligar corredor    // <- falta ';'
        esperar 10s;
        desligar corredor;
    }

    modo restart;
}
```

Saída do compilador:

```
$ python3 src/main.py examples/erro_sintatico_multiplos.homi
Encontrei 3 problemas de sintaxe.
- Linha 2, coluna 0: faltou ';' antes de 'entidade'
- Linha 7, coluna 4: faltou ';' antes de 'faca'
- Linha 9, coluna 8: faltou ';' antes de 'esperar'
```


9.4. Erro Semântico — erro_semantico.homi

Ação ligar aplicada a uma entidade do domínio sensor, que não é acionável:

```
entidade sensor movimento = binary_sensor.movimento;
entidade luz corredor = light.corredor;

automacao "Erro semantico" {
    quando estado movimento ficar on;

    faca {
        ligar movimento;    // ERRO: sensor nao pode ser ligado
    }

    modo restart;
}
```

Saída do compilador:

```
$ python3 src/main.py examples/erro_semantico.homi
Sintaxe ok.
Encontrei 1 problema semantico.
- Linha 8: acao 'ligar' nao pode ser aplicada a entidade 'movimento',
  pois ela foi declarada como 'sensor'.
```

9.5. Erro Semântico em Condição — erro_semantico_condicao.homi

Uso de operadores 'e' e 'ou' misturados na mesma expressaoCondicao, e negação com NAO:

```
entidade sensor movimento = binary_sensor.movimento;
entidade luz corredor = light.corredor;
entidade interruptor tomada = switch.tomada;

automacao "Erro semantico condicao" {
    quando estado movimento ficar on;
    se estado corredor ficar off
    e estado tomada ficar off
    ou nao estado movimento ficar off;    // ERRO: mistura e/ou

    faca {
        ligar corredor;
    }

    modo restart;
}
```

Saída do compilador:

```
$ python3 src/main.py examples/erro_semantico_condicao.homi
Sintaxe ok.
```

Encontrei 1 problema semantico.

- Linha 7: condicao nao pode misturar operadores 'e' e 'ou' na mesma expressao.

10. Estrutura de Arquivos do Projeto

```
.
├── grammar/
│   └── Homi.g4                                # Gramatica completa (lexer + parser
ANTLR)
├── generated/
│   └── grammar/                                # Arquivos gerados pelo ANTLR (nao
editar)
│       ├── HomiLexer.py
│       ├── HomiParser.py
│       ├── HomiVisitor.py
│       └── HomiListener.py
├── src/
│   ├── main.py                                # Ponto de entrada; orchestra o pipeline
│   └── table_parser.py                        # Parser LL(1) por tabela preditiva +
build_ast
│   ├── ast_nodes.py                          # Nos da AST (dataclasses)
│   ├── ast_builder.py                        # Visitor ANTLR alternativo
│   ├── semantic.py                          # Analise semantica + tabela de simbolos
│   └── yaml_generator.py                     # Geracao de YAML para Home Assistant
├── examples/
│   ├── exemplo1.homi                         # Movimento + condicao + modo restart
│   ├── exemplo2.homi                         # Tablet carregando (sensor charging)
│   ├── exemplo3.homi                         # Alarme desarmado (condicao AND + NAO)
│   ├── exemplo4.homi                         # Lavagem finalizada (entity_id direto)
│   └── exemplo5.homi                         # Cozinha ocupada (condicao OR + modo
parallel)
│       ├── erro_sintatico.homi
│       ├── erro_sintatico_multiplos.homi
│       ├── erro_semantico.homi
│       └── erro_semantico_condicao.homi
├── outputs/                                   # YAMLs gerados (criado automaticamente)
├── antlr-4.13.2-complete.jar                 # ANTLR incluido no projeto
├── Makefile                                  # Atalhos: generate, run-all, test, clean
├── requirements.txt                           # antlr4-python3-runtime==4.13.2
└── README.md
```

11. Instruções de Execução

11.1. Instalação

```
python3 -m venv .venv
source .venv/bin/activate          # Linux/macOS
# .venv\Scripts\activate           # Windows
pip install -r requirements.txt    # antlr4-python3-runtime==4.13.2
```

11.2. Regenerar o lexer (após editar Homi.g4)

```
make generate
# equivalente a:
java -jar antlr-4.13.2-complete.jar -Dlanguage=Python3 -visitor \
    grammar/Homi.g4 -o generated/grammar
```

11.3. Compilar um arquivo .homi

```
python3 src/main.py examples/exemplo1.homi
# O YAML é gerado em outputs/
```

11.4. Executar todos os exemplos válidos

```
make run-all
```

11.5. Executar os casos de teste de erro

```
make test
# Executa os quatro arquivos de erro:
# erro_sintatico.homi, erro_sintatico_multiplos.homi,
# erro_semantico.homi, erro_semantico_condicao.homi
```

11.6. Limpar arquivos gerados

```
make clean # remove outputs/*.yaml
```

12. Considerações Finais

O compilador Homi demonstra, de forma funcional e didática, a aplicação prática dos conceitos fundamentais da teoria de compiladores: análise léxica baseada em autômatos finitos e expressões regulares via ANTLR, análise sintática descendente LL(1) com tabela preditiva construída manualmente, construção de árvore de sintaxe abstrata por segunda passagem sobre os tokens, análise semântica com tabela de símbolos e verificação de tipos, e geração de código-alvo por percurso da AST validada.

O ponto técnico mais relevante do trabalho é a implementação manual do parser LL(1) em `table_parser.py`. A construção da tabela preditiva exigiu o cálculo dos conjuntos FIRST e FOLLOW da gramática, a eliminação de ambiguidades e a verificação da propriedade LL(1) para cada produção. O parser realiza duas passagens sobre os tokens: a primeira para validar a sintaxe com recuperação de erros em modo pânico, e a segunda para construir a AST diretamente.

A presença do `ast_builder.py` demonstra uma segunda abordagem para construção de AST usando o padrão Visitor do ANTLR sobre a parse tree, evidenciando o contraste entre a abordagem automática e a manual.

A adoção da técnica de Modo Pânico na recuperação de erros sintáticos, combinada com o acúmulo de erros semânticos, confere ao sistema robustez e uma experiência de diagnóstico significativamente superior à de compiladores que interrompem na primeira falha. A aplicação a um domínio concreto — automação residencial com Home Assistant — evidenciou as vantagens práticas das DSLs: simplicidade de uso, garantia de validade por análise semântica e geração automática de configuração correta.

Como extensões naturais para versões futuras: suporte a gatilhos por horário, condições compostas com agrupamento explícito via parênteses, variáveis de estado, múltiplos gatilhos por automação e suporte a novos domínios como `climate` e `media_player`.

Dependência: `antlr4-python3-runtime==4.13.2` | Java 8+ (para regenerar o `lexer`)

Relatório gerado para fins acadêmicos — Disciplina de Compiladores.